01 · Data Collection — Web Scraping MagicBricks

The data for this project was collected by scraping residential sale listings from MagicBricks, an Indian property marketplace.

Tech stack: Python · Selenium (renders the JavaScript-heavy pages) · BeautifulSoup (parses the HTML) · Pandas (stores the results)

Output: `data/raw/MagicBricks_Raw.xlsx` — 2,989 listings × 27 columns across 59 Indian cities.

> The scraper lives in `src/scraper.py` so it can be run from the command line. It needs Chrome and live internet access, so this notebook explains it and checks the parser offline instead of re-scraping. Re-running it today may return different listings, and the site's HTML class names may have changed since the data was collected.

How the scraper works

1. Build the search URL for each `city` and `page`.
2. Load it in a Chrome browser controlled by Selenium, then wait a randomised 2–4 seconds so we don't overload the site.
3. Parse the page with BeautifulSoup and find every listing card (`div.mb-srp__list`).
4. For each card, collect:
   - **Card headers:** title, society name and price.
   - **Hidden `<meta itemprop=…>` tags:** description, URL, locality, latitude and longitude.
   - **Summary label/value pairs:** Super Area, Carpet Area, Status, Floor, Transaction, Furnishing, Bathroom, Car Parking and so on.
5. Save all records to a timestamped Excel file in `data/raw/` .

Run it from the project root:

```
python src/scraper.py --cities Pune Mumbai --pages 2
```

```
import sys
sys.path.append('../src')
from bs4 import BeautifulSoup
from scraper import MagicBricksScraper, BASE_URL
```

```
print(BASE_URL.format(page=1, city='Pune'))
```

```
https://www.magicbricks.com/property-for-sale/residential-real-estate?proptype=Multistorey-Apartment,Builder-Floor-Apartment,Penthouse,Studio-Apartment,Residential-House,Villa&page=1&cityName=Pune
```

Offline test of the parser

To check that the parsing logic works without hitting the live site, we run `parse_card` on a sample listing card that mirrors MagicBricks' HTML structure.

```
sample_card = '''
<div class="mb-srp__list" id="cardid12345678">
  <meta itemprop="description" content="2 Bath, Semi-Furnished flat in Baner">
  <meta itemprop="url" content="https://www.magicbricks.com/propertyDetails/sample">
  <meta itemprop="addressLocality" content="Baner">
  <meta itemprop="latitude" content="18.5590">
  <meta itemprop="longitude" content="73.7868">
  <h2 class="mb-srp__card--title">2 BHK Flat for Sale in Baner, Pune</h2>
  <a class="mb-srp__card__society--name">Green Valley Residency</a>
  <div class="mb-srp__card__price--amount">&#8377;85 Lac</div>
  <div class="mb-srp__card__summary--label">Super Area</div><div class="mb-srp__card__summary--value">1050 sqft</div>
  <div class="mb-srp__card__summary--label">Status</div><div class="mb-srp__card__summary--value">Ready to Move</div>
  <div class="mb-srp__card__summary--label">Transaction</div><div class="mb-srp__card__summary--value">Resale</div>
  <div class="mb-srp__card__summary--label">Bathroom</div><div class="mb-srp__card__summary--value">2</div>
</div>'''

card = BeautifulSoup(sample_card, 'html.parser').find('div', 'mb-srp__list')
record = MagicBricksScraper.parse_card(card, 'Pune')
record
```

```
{'Property Id': 'cardid12345678',
 'Title': '2 BHK Flat for Sale in Baner, Pune',
 'Society Name': 'Green Valley Residency',
 'Price': '₹85 Lac',
 'City': 'Pune',
 'Description': '2 Bath, Semi-Furnished flat in Baner',
 'URL': 'https://www.magicbricks.com/propertyDetails/sample',
 'Address': 'Baner',
 'Latitude': '18.5590',
 'Longitude': '73.7868',
 'Super Area': '1050 sqft',
 'Status': 'Ready to Move',
 'Transaction': 'Resale',
 'Bathroom': '2'}
```

```
expected = {'Property Id': 'cardid12345678', 'Price': '₹85 Lac', 'Address': 'Baner',
            'Super Area': '1050 sqft', 'Transaction': 'Resale', 'City': 'Pune'}
assert all(record[k] == v for k, v in expected.items()), 'Parser output does not match'
print('Parser test passed ✔')
```

```
Parser test passed ✔
```

The collected raw dataset

```
import pandas as pd
raw = pd.read_excel('../data/raw/MagicBricks_Raw.xlsx')
print(raw.shape)
print('Cities covered:', raw['City'].nunique())
raw.head(3)
```

```
(2989, 27)
Cities covered: 59
```

	Property Id	Title	Society Name	Price	Name	Description	URL	Address	City	Latitude	...	Society	Car Parking	Bathroom	Cons
0	cardid51696257	4 BHK Flat for Sale Luxury Floors, in DLF City...	Luxury Floors	₹5 Cr	4 BHK Flat for Sale Luxury Floors, in DLF C...	4 Bath,Semi-Furnished,4 floor Miglani Luxury F...	https://www.magicbricks.com/propertyDetails/4-...	DLF City Phase 1	Gurgaon	77.098466	...	Luxury Floors	1 Covered	4	
1	cardid57200639	3 BHK Flat for Sale Eldeco Acclaim, in Sohna	Eldeco Acclaim	₹92.8 Lac	3 BHK Flat for Sale Eldeco Acclaim, in Sohna	3 Bath,Unfurnished Sprawled over 13 acres of v...	https://www.magicbricks.com/propertyDetails/3-...	Sohna	Gurgaon	77.069185	...	Eldeco Acclaim	NaN	3	
2	cardid52514647	2 BHK Flat for Sale The Leaf, in Sector 85	The Leaf	₹1.18 Cr	2 BHK Flat for Sale The Leaf, in Sector 85	2 Bath,Semi-Furnished Welcome to nature's lap ...	https://www.magicbricks.com/propertyDetails/2-...	Sector 85	Gurgaon	76.953027	...	The Leaf	NaN	2	

3 rows × 27 columns

02 · Data Cleaning & Preprocessing

Turns the raw scraped listings into an analysis-ready table.

Steps

1. Overview and missing values
2. Drop unwanted columns
3. Extract numbers from text: bedrooms, price and area
4. Fix data errors: invalid bathrooms and swapped latitude/longitude
5. Add features: property type, parking, state, city tier and price per sqft
6. Remove duplicates, implausible listings and city-level price outliers
7. Save the cleaned dataset

```
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', 30)
sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 110

ROOT = Path('..')
IMG_DIR = ROOT / 'images'
IMG_DIR.mkdir(exist_ok=True)

def save(fig, name):
    fig.tight_layout()
    fig.savefig(IMG_DIR / f'{name}.png', bbox_inches='tight')
    plt.show()
```

1. Overview of the dataset

```
housing = pd.read_excel(ROOT / 'data/raw/MagicBricks_Raw.xlsx')
print(housing.shape)
housing.head(3)
```

```
(2989, 27)
```

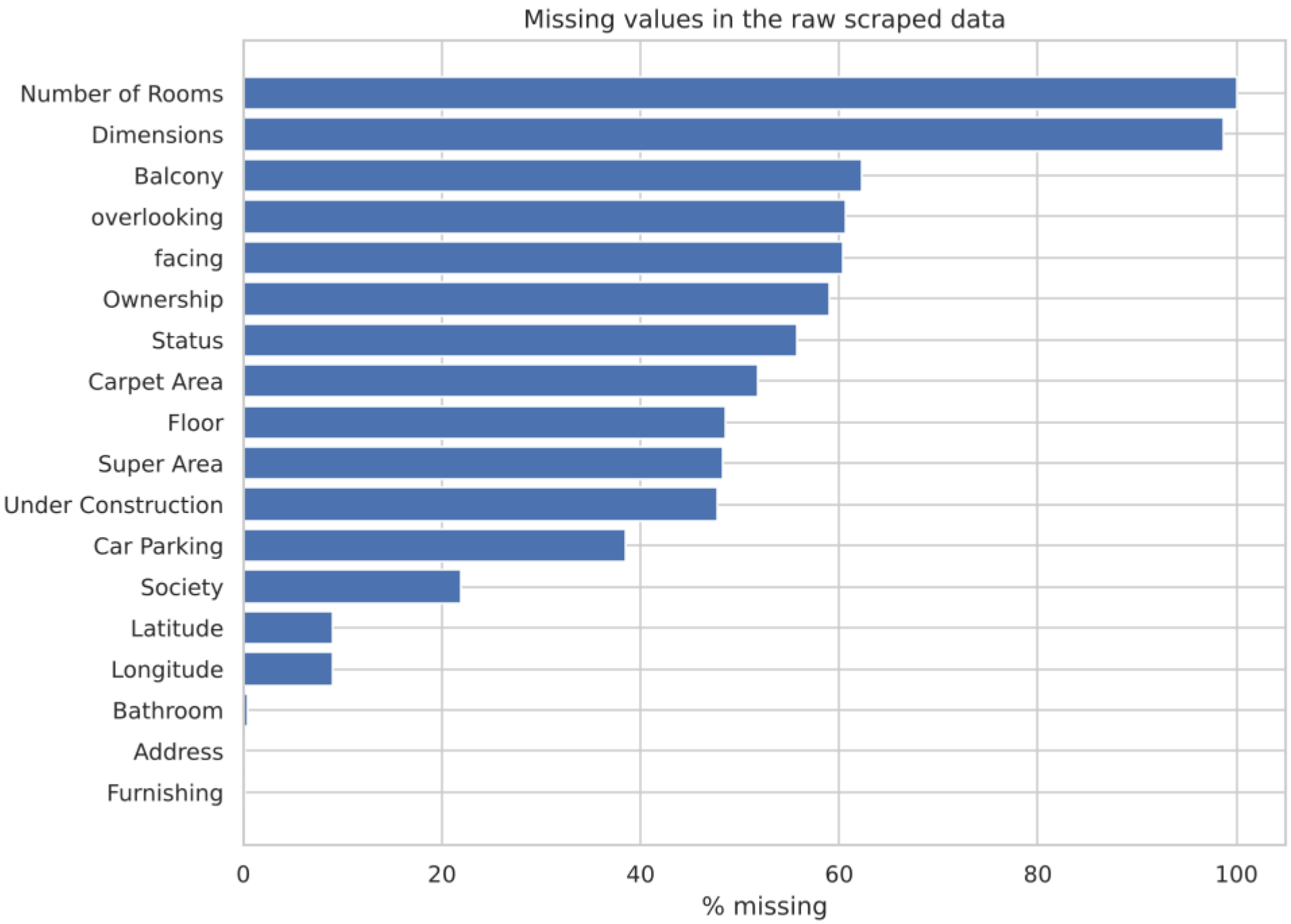
	Property Id	Title	Society Name	Price	Name	Description	URL	Address	City	Latitude	Longitude	Number of Rooms	Super Area	Status	F
0	cardid51696257	4 BHK Flat for Sale Luxury Floors, in DLF City...	Luxury Floors	₹5 Cr	4 BHK Flat for Sale Luxury Floors, in DLF C...	4 Bath,Semi-Furnished,4 floor Miglani Luxury F...	https://www.magicbricks.com/propertyDetails/4-...	DLF City Phase 1	Gurgaon	77.098466	28.472934	NaN	4500 sqft	Ready to Move	
1	cardid57200639	3 BHK Flat for Sale Eldeco Acclaim, in Sohna	Eldeco Acclaim	₹92.8 Lac	3 BHK Flat for Sale Eldeco Acclaim, in Sohna	3 Bath,Unfurnished Sprawled over 13 acres of v...	https://www.magicbricks.com/propertyDetails/3-...	Sohna	Gurgaon	77.069185	28.247592	NaN	1751 sqft	Ready to Move	
2	cardid52514647	2 BHK Flat for Sale The Leaf, in Sector 85	The Leaf	₹1.18 Cr	2 BHK Flat for Sale The Leaf, in Sector 85	2 Bath,Semi-Furnished Welcome to nature's lap ...	https://www.magicbricks.com/propertyDetails/2-...	Sector 85	Gurgaon	76.953027	28.403726	NaN	1741 sqft	Ready to Move	

```
missing = (housing.isna().mean() * 100).round(2).sort_values(ascending=False)
missing.to_frame('% missing')
```

	% missing
Number of Rooms	100.00
Dimensions	98.70
Balcony	62.26
overlooking	60.62
facing	60.42
Ownership	58.98
Status	55.80
Carpet Area	51.86
Floor	48.51
Super Area	48.31
Under Construction	47.67
Car Parking	38.54
Society	21.91
Latitude	8.97
Longitude	8.97
Bathroom	0.50
Address	0.17
Furnishing	0.10
Society Name	0.00
City	0.00
Name	0.00
Description	0.00
URL	0.00
Price	0.00
Property Id	0.00
Title	0.00
Transaction	0.00

Observation: fields shown on every listing card (title, price, city, transaction) are almost complete. Optional fields such as balcony, facing and ownership are 40–60% empty. `Number of Rooms` is completely empty, and `Dimensions` is 98.7% empty.

```
fig, ax = plt.subplots(figsize=(9, 6.5))
m = missing[missing > 0]
ax.barh(m.index, m.values, color='#4C72B0'); ax.invert_yaxis()
ax.set_xlabel('% missing'); ax.set_title('Missing values in the raw scraped data')
save(fig, 're_01_missing_values')
```


Missing values in the raw scraped data
Number of Rooms
Dimensions
Balcony
overlooking
facing
Ownership
Status
Carpet Area
Floor
Super Area
Under Construction
Car Parking
Society
Latitude
Longitude
Bathroom
Address
Furnishing
0
20
40
60
80
100
% missing

2. Drop unwanted columns

- `Name` repeats `Title`
- `Number of Rooms` and `Dimensions` are almost completely empty
- `Under Construction` repeats the information in `Status`

```
housing = housing.drop(columns=['Name', 'Number of Rooms', 'Dimensions', 'Under Construction'])
housing['Status'] = housing['Status'].fillna('Under Construction')
housing['Status'].value_counts()
```

```
Status
Under Construction    1668
Ready to Move         1321
Name: count, dtype: int64
```

3. Extract numbers from text

3.1 Bedrooms and property type from `Title`

Titles look like `"3 BHK Flat for Sale ..."` or `"Studio Apartment for Sale ..."`. A studio is treated as 1 bedroom. The single `"> 10 BHK House"` listing can't be given a number, so it is dropped later.

```
title = housing['Title'].str.strip()
housing['Bedrooms'] = pd.to_numeric(title.str.extract(r'^(\d+)\s*BHK', expand=False))
housing.loc[title.str.startswith('Studio'), 'Bedrooms'] = 1

ptype = title.str.extract(r'^(?:\d+\s*BHK\s+|Studio\s+)?(.*?)\s+for Sale', expand=False)
housing['Property Type'] = ptype.replace({'Apartment': 'Studio Apartment'})
housing.loc[title.str.startswith('Studio'), 'Property Type'] = 'Studio Apartment'
housing[['Title', 'Bedrooms', 'Property Type']].head()
```

	Title	Bedrooms	Property Type
0	4 BHK Flat for Sale Luxury Floors, in DLF City...	4.0	Flat
1	3 BHK Flat for Sale Eldeco Acclaim, in Sohna	3.0	Flat
2	2 BHK Flat for Sale The Leaf, in Sector 85	2.0	Flat
3	Studio Apartment for Sale M3M Corner Walk, in...	1.0	Studio Apartment
4	2 BHK Flat for Sale M3M Soulitude, in Sector 89	2.0	Flat

```
housing['Property Type'].value_counts()
```

```
Property Type
Flat                2698
Villa                164
Studio Apartment      63
House                 63
> 10 BHK House         1
Name: count, dtype: int64
```

3.2 Price in rupees

Prices look like `₹92.8 Lac` or `₹1.18 Cr`. 1 Lac = 1,00,000 and 1 Cr = 1,00,00,000. "Call for Price" listings have no price and become NaN.

```
def parse_price(text):
    m = pd.Series(text).astype(str).str.extract(r'₹\s*([\d.]+)\s*(Lac|Cr)')
    value = pd.to_numeric(m[0])
    return np.where(m[1] == 'Cr', value * 1e7, value * 1e5)

housing['Price'] = parse_price(housing['Price'])
housing['Price'].describe()
```

```
count    2.985000e+03
mean     6.001546e+07
std      1.165611e+09
min      1.000000e+05
25%      4.570000e+06
50%      6.950000e+06
75%      1.200000e+07
max      4.118000e+10
Name: Price, dtype: float64
```

3.3 Area in square feet

`Super Area` is used where it exists, with `Carpet Area` as the fallback. Units are converted to sqft:

- 1 sq yard = 9 sqft
- 1 sq metre = 10.764 sqft

```
area_text = housing['Super Area'].fillna(housing['Carpet Area']).astype(str)
parts = area_text.str.extract(r'([\d.]+)\s*(sqft|sqyrd|sqm)')
```

```
factor = parts[1].map({'sqft': 1, 'sqyrd': 9, 'sqm': 10.764})
housing['Area(sqft)'] = pd.to_numeric(parts[0]) * factor
print(parts[1].value_counts(dropna=False))
housing['Area(sqft)'].describe()
```

```
1
sqft     2916
sqyrd      50
sqm        18
NaN         5
Name: count, dtype: int64
```

```
count     2984.000000
mean      1319.898984
std       1235.840550
min        173.000000
25%        737.000000
50%       1120.000000
75%       1600.000000
max      50000.000000
Name: Area(sqft), dtype: float64
```

4. Fix data errors

4.1 Invalid bathroom values

```
print(housing['Bathroom'].value_counts(dropna=False))
housing['Bathroom'] = pd.to_numeric(housing['Bathroom'], errors='coerce')  # '> 10' -> NaN
```

```
Bathroom
2       1560
3        844
4        236
1        224
5         80
6         23
NaN       15
7          6
> 10       1
Name: count, dtype: int64
```

4.2 Latitude and longitude are swapped

India lies between roughly **6–37° N latitude** and **68–98° E longitude**. In the raw data, the `Latitude` column holds values around 72–88, which are longitudes. The scraper stored the two fields the wrong way round, so we swap them back and check the result.

```
print('Before:', housing[['Latitude', 'Longitude']].describe().loc[['min', 'max']].round(2).to_dict())
housing[['Latitude', 'Longitude']] = housing[['Longitude', 'Latitude']].values
ok = housing['Latitude'].between(6, 37) & housing['Longitude'].between(68, 98)
print('Rows with valid India coordinates:', ok.sum(), 'of', housing['Latitude'].notna().sum())
```

```
Before: {'Latitude': {'min': 26.89, 'max': 91.82}, 'Longitude': {'min': 8.49, 'max': 30.71}}
Rows with valid India coordinates: 2719 of 2721
```

5. Feature engineering

5.1 Parking spaces and furnishing

```
# '2 Covered, 1 Open' -> 3 spaces; missing -> 0 (not specified)
housing['Parking'] = (housing['Car Parking'].fillna('').astype(str).str.findall(r'(\d+)')
                      .apply(lambda xs: sum(int(x) for x in xs)))
# A few listings give the whole society's parking (e.g. '835 Covered'), not the flat's.
print('Listings with more than 10 parking spaces (society totals):', (housing['Parking'] > 10).sum())
housing.loc[housing['Parking'] > 10, 'Parking'] = 0
housing['Furnishing'] = housing['Furnishing'].fillna('Unfurnished')
housing['Parking'].value_counts().sort_index().head(8)
```

```
Listings with more than 10 parking spaces (society totals): 8
```

```
Parking
0    1160
1    1551
2     223
3      33
4      11
5       3
6       4
7       2
Name: count, dtype: int64
```

5.2 State for each city

```python
state_map = {
    'Kerala': ['Kochi', 'Trivandrum', 'Thrissur'],
    'Maharashtra': ['Aurangabad', 'Nashik', 'Mumbai', 'Navi-Mumbai', 'Pune', 'Bhiwandi', 'Thane', 'Nagpur'],
    'Gujarat': ['Ahmedabad', 'Surat', 'Vadodara', 'Gandhinagar', 'Vapi'],
    'Karnataka': ['Mangalore', 'Bangalore', 'Mysore'],
    'Uttar Pradesh': ['Meerut', 'Agra', 'Allahabad', 'Kanpur', 'Varanasi', 'Vrindavan', 'Noida', 'Greater-Noida',
                      'Ghaziabad', 'Lucknow'],
    'Telangana': ['Hyderabad'],
    'Haryana': ['Gurgaon', 'Faridabad', 'Sonipat'],
    'Tamil Nadu': ['Chennai', 'Coimbatore', 'Trichy'],
    'West Bengal': ['Durgapur', 'Kolkata'],
    'Rajasthan': ['Jodhpur', 'Jaipur', 'Udaipur'],
    'Madhya Pradesh': ['Bhopal', 'Indore'],
    'Odisha': ['Bhubaneswar'],
    'Andhra Pradesh': ['Tirupati', 'Vijayawada', 'Visakhapatnam', 'Guntur'],
    'Assam': ['Guwahati'],
    'Jharkhand': ['Bokaro-Steel-City', 'Ranchi', 'Jamshedpur'],
    'Uttarakhand': ['Haridwar', 'Dehradun'],
    'Chhattisgarh': ['Raipur'],
    'Bihar': ['Patna'],
    'Delhi': ['New-Delhi'],
    'Goa': ['Goa'],
    'Chandigarh': ['Chandigarh'],
}
city_to_state = {city: state for state, cities in state_map.items() for city in cities}
housing['State/Territory'] = housing['City'].map(city_to_state)
assert housing['State/Territory'].notna().all(), housing.loc[housing['State/Territory'].isna(), 'City'].unique()
housing['State/Territory'].value_counts().head(10)
```

```
State/Territory
Maharashtra      873
Uttar Pradesh    482
Karnataka        253
Gujarat          253
Haryana          196
Tamil Nadu       158
Telangana        150
Rajasthan        109
West Bengal       99
Kerala            62
Name: count, dtype: int64
```

5.3 City tier

Cities are grouped into tiers based on India's HRA city classification:

- **Tier 1:** X-class metros
- **Tier 2:** Y-class cities
- **Tier 3 and Tier 4:** smaller Z-class cities

The original code had cities appearing in two lists and silently labelled any unmatched city as Tier 1. Each city now has exactly one tier, and an assert catches any city that's missing.

```
tiers = {
    'Tier1': ['Kolkata', 'Chennai', 'Hyderabad', 'Bangalore', 'Navi-Mumbai', 'Mumbai', 'New-Delhi'],
    'Tier2': ['Ahmedabad', 'Greater-Noida', 'Goa', 'Jamshedpur', 'Bokaro-Steel-City', 'Indore', 'Guntur', 'Thrissur',
              'Allahabad', 'Jaipur', 'Patna', 'Gandhinagar', 'Bhopal', 'Pune', 'Lucknow', 'Gurgaon', 'Ranchi', 'Thane',
              'Aurangabad', 'Kochi', 'Nagpur', 'Mangalore', 'Faridabad', 'Durgapur', 'Kanpur', 'Dehradun', 'Varanasi',
              'Noida', 'Surat', 'Raipur', 'Tirupati', 'Chandigarh', 'Trichy', 'Vadodara', 'Nashik', 'Mysore',
              'Ghaziabad', 'Visakhapatnam', 'Bhiwandi', 'Meerut'],
    'Tier3': ['Coimbatore', 'Haridwar', 'Agra', 'Udaipur', 'Vijayawada', 'Sonipat'],
    'Tier4': ['Vrindavan', 'Guwahati', 'Bhubaneswar', 'Trivandrum', 'Vapi', 'Jodhpur'],
}
city_to_tier = {city: tier for tier, cities in tiers.items() for city in cities}
housing['Tier'] = housing['City'].map(city_to_tier)
assert housing['Tier'].notna().all(), housing.loc[housing['Tier'].isna(), 'City'].unique()
housing['Tier'].value_counts()
```

```
Tier
Tier2    1858
Tier1     914
Tier4     121
Tier3      96
Name: count, dtype: int64
```

6. Remove duplicates and invalid rows

```
n0 = len(housing)
housing = housing.drop_duplicates(subset='Property Id')
print(f'Duplicate listings removed: {n0 - len(housing)}')
```

```
housing = housing.dropna(subset=['Price', 'Area(sqft)', 'Bathroom', 'Bedrooms'])
print('After dropping rows missing price/area/bathroom/bedrooms:', len(housing))
```

```
Duplicate listings removed: 932
After dropping rows missing price/area/bathroom/bedrooms: 2039
```

```
housing['Price per sqft'] = housing['Price'] / housing['Area(sqft)']
housing['Price per sqft'].describe().round(0)
```

```
count        2039.0
mean        22797.0
std        443204.0
min            56.0
25%          4503.0
50%          6400.0
75%         10126.0
max      17194154.0
Name: Price per sqft, dtype: float64
```

Some listings are clearly data-entry errors, such as a 1-sqft flat or a price of ₹50/sqft. We keep only plausible values:

- Area: 150–15,000 sqft
- Price per sqft: ₹1,000–₹1,00,000

```
valid = housing['Area(sqft)'].between(150, 15000) & housing['Price per sqft'].between(1000, 100000)
print('Implausible rows removed:', (~valid).sum())
print(housing.loc[~valid, ['Title', 'Price', 'Area(sqft)', 'Price per sqft']].head())
housing = housing[valid].copy()
```

```
Implausible rows removed: 6
                                                  Title         Price  \
236                   2 BHK Flat for Sale in Sector 117   3.400000e+06
834       2 BHK Flat for Sale Lodha The Park, in Worli   1.200000e+05
1545                   5 BHK Villa for Sale in Perungudi  3.900000e+07
2483  2 BHK House for Sale in Gruham Garden Valley, ...  1.000000e+05
2512  4 BHK Flat for Sale Viaana, in Vijayanagar 4th...  4.118000e+10

      Area(sqft)  Price per sqft
236       5994.0    5.672339e+02
834        890.0    1.348315e+02
1545     50000.0    7.800000e+02
2483      1800.0    5.555556e+01
2512      2395.0    1.719415e+07
```

Some listings are far from their own city's typical price, such as a 2 BHK in Trivandrum at ₹60,000/sqft or a 1 BHK in Gurgaon at ₹45,000/sqft. These are usually typos (₹95 Lac entered as ₹9.5 Cr) or mislabelled properties.

Like the original `remove_pps_outliers` step, we drop listings whose price per sqft is **more than 3× or less than ⅓ of their city's median**. This only applies to cities with at least 5 listings.

```
city_median = housing.groupby('City')['Price per sqft'].transform('median')
city_count = housing.groupby('City')['City'].transform('size')
ratio = housing['Price per sqft'] / city_median
city_outlier = (city_count >= 5) & ((ratio > 3) | (ratio < 1 / 3))
print(f'City-level price outliers removed: {city_outlier.sum()} ({city_outlier.mean():.1%})')
housing = housing[~city_outlier].copy()
```

```
City-level price outliers removed: 29 (1.4%)
```

7. Final column order and save

```
cols = ['Property Id', 'Society Name', 'Property Type', 'Bedrooms', 'Bathroom', 'Area(sqft)', 'Parking',
        'Address', 'City', 'Tier', 'State/Territory', 'Latitude', 'Longitude', 'Transaction', 'Furnishing',
        'Status', 'Description', 'URL', 'Price per sqft', 'Price']
housing = housing[cols].reset_index(drop=True)
housing[['Bedrooms', 'Bathroom', 'Parking']] = housing[['Bedrooms', 'Bathroom', 'Parking']].astype(int)
housing['Price per sqft'] = housing['Price per sqft'].round(0).astype(int)
housing.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2004 entries, 0 to 2003
Data columns (total 20 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Property Id      2004 non-null   str
 1   Society Name     2004 non-null   str
 2   Property Type    2004 non-null   str
 3   Bedrooms         2004 non-null   int64
 4   Bathroom         2004 non-null   int64
 5   Area(sqft)       2004 non-null   float64
 6   Parking          2004 non-null   int64
 7   Address          1999 non-null   str
 8   City             2004 non-null   str
 9   Tier             2004 non-null   str
 10  State/Territory  2004 non-null   str
 11  Latitude         1833 non-null   float64
 12  Longitude        1833 non-null   float64
 13  Transaction      2004 non-null   str
 14  Furnishing       2004 non-null   str
 15  Status           2004 non-null   str
 16  Description      2004 non-null   str
 17  URL              2004 non-null   str
 18  Price per sqft   2004 non-null   int64
 19  Price            2004 non-null   float64
dtypes: float64(4), int64(4), str(12)
memory usage: 313.3 KB
```

```python
out = ROOT / 'data/processed/magicbricks_cleaned.csv'
housing.to_csv(out, index=False)
print(f'Saved {len(housing)} clean listings ({len(housing) / 2989:.0%} of raw) to {out}')
```

```
Saved 2004 clean listings (67% of raw) to ../data/processed/magicbricks_cleaned.csv
```

03 · Exploratory Data Analysis

Questions explored:

1. What do the property price and size distributions look like?
2. How do prices differ by city, state and city tier?
3. How do bedrooms, furnishing and new vs. resale affect price?
4. Which features are most related to price?

```
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', 30)
sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 110

ROOT = Path('..')
IMG_DIR = ROOT / 'images'
IMG_DIR.mkdir(exist_ok=True)

def save(fig, name):
    fig.tight_layout()
    fig.savefig(IMG_DIR / f'{name}.png', bbox_inches='tight')
    plt.show()

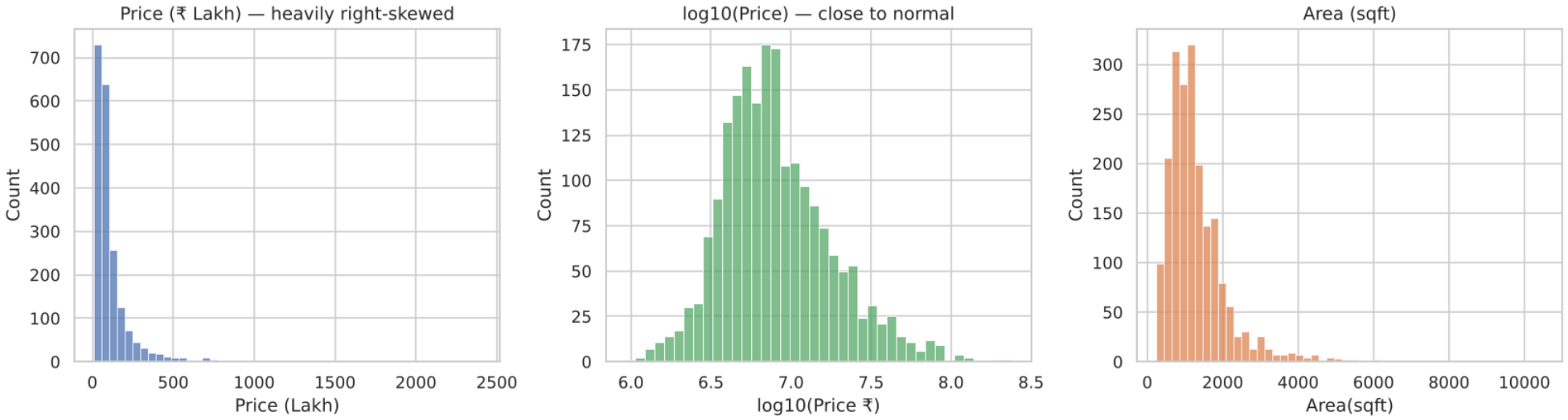
df = pd.read_csv(ROOT / 'data/processed/magicbricks_cleaned.csv')
df['Price (Lakh)'] = df['Price'] / 1e5
print(df.shape)
df.describe().round(1).T
```

```
(2004, 21)
```

	count	mean	std	min	25%	50%	75%	max
Bedrooms	2004.0	2.5	0.9	1.0	2.0	2.0	3.0	7.0
Bathroom	2004.0	2.5	0.9	1.0	2.0	2.0	3.0	7.0
Area(sqft)	2004.0	1310.7	841.8	242.0	774.8	1129.5	1575.8	10500.0
Parking	2004.0	0.7	0.8	0.0	0.0	1.0	1.0	10.0
Latitude	1833.0	21.5	5.7	8.5	18.5	19.3	28.4	30.7
Longitude	1833.0	76.7	3.8	26.9	73.3	77.2	77.7	91.8
Price per sqft	2004.0	9182.7	8470.2	1761.0	4504.8	6375.0	10000.0	65385.0
Price	2004.0	11607425.1	14685328.0	920000.0	4680000.0	7170000.0	12725000.0	240000000.0
Price (Lakh)	2004.0	116.1	146.9	9.2	46.8	71.7	127.2	2400.0

1. Distributions

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
sns.histplot(df['Price (Lakh)'], bins=50, ax=axes[0], color='#4C72B0')
axes[0].set_title('Price (₹ Lakh) — heavily right-skewed')
sns.histplot(np.log10(df['Price']), bins=40, ax=axes[1], color='#55A868')
axes[1].set_title('log10(Price) — close to normal'); axes[1].set_xlabel('log10(Price ₹)')
sns.histplot(df['Area(sqft)'], bins=50, ax=axes[2], color='#DD8452')
axes[2].set_title('Area (sqft)')
save(fig, 're_02_distributions')
```



Observation: prices are strongly right-skewed. The median is about ₹72 Lakh, but there is a long tail of luxury properties worth several crores. On a log scale the distribution is roughly normal, which is why the model predicts **log(price)**.

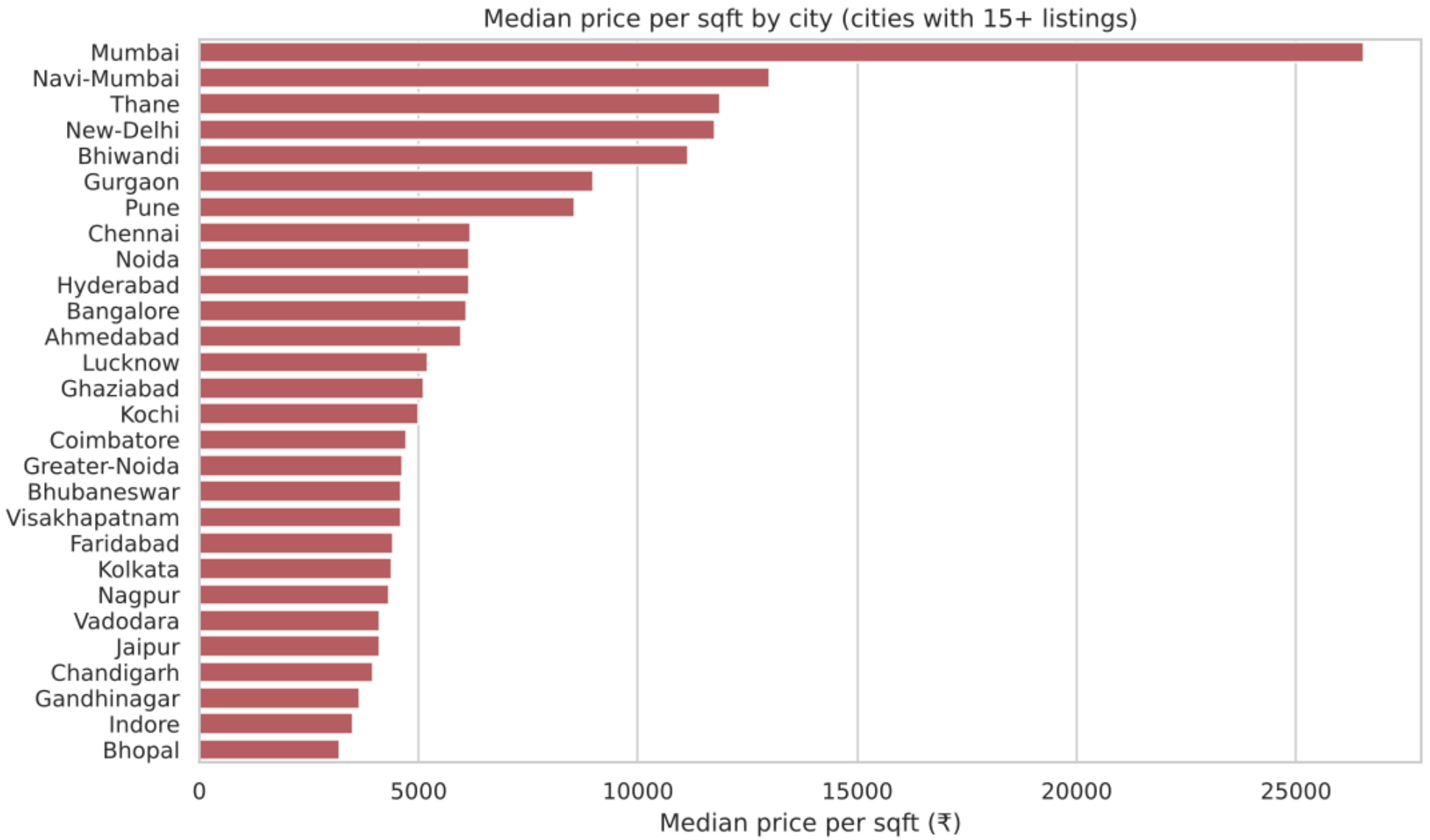
```
bed_counts = df['Bedrooms'].value_counts().sort_index()
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(x=bed_counts.index, y=bed_counts.values, ax=ax, color='#8172B3')
ax.set_xlabel('Bedrooms (BHK)'); ax.set_ylabel('Listings'); ax.set_title('Listings by number of bedrooms')
save(fig, 're_03_bedroom_counts')
```



Observation: most listings are 2 BHK and 3 BHK flats, so the model will be most reliable for these.

2. Location

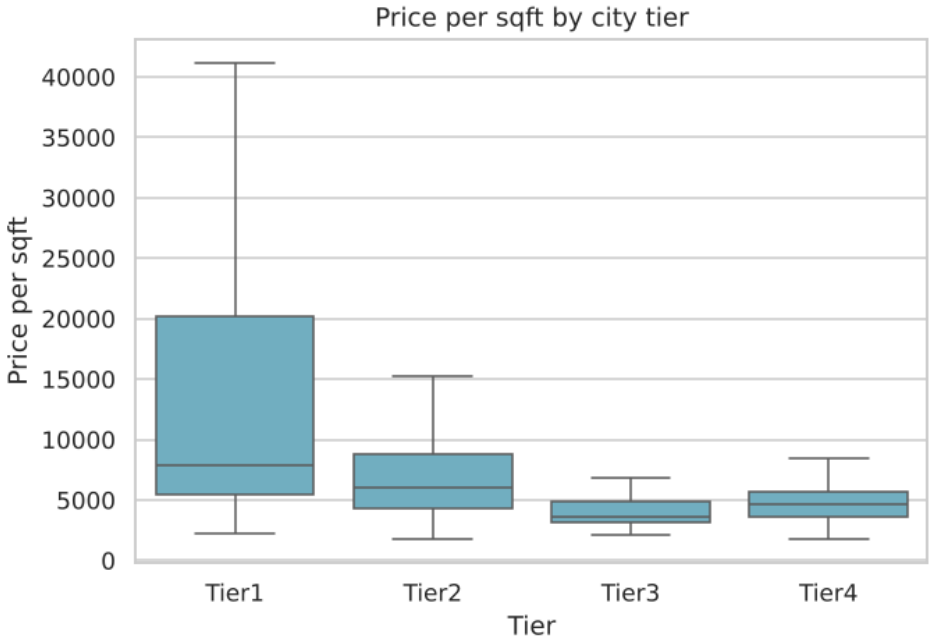
```
city_stats = (df.groupby('City').agg(listings=('Price', 'size'),
                                     median_price_lakh=('Price (Lakh)', 'median'),
                                     median_pps=('Price per sqft', 'median'))
              .query('listings >= 15').sort_values('median_pps', ascending=False))
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(x=city_stats['median_pps'], y=city_stats.index, ax=ax, color='#C44E52')
ax.set_xlabel('Median price per sqft (₹)'); ax.set_ylabel('')
ax.set_title('Median price per sqft by city (cities with 15+ listings)')
save(fig, 're_04_city_price_per_sqft')
city_stats.round(0).head(10)
```

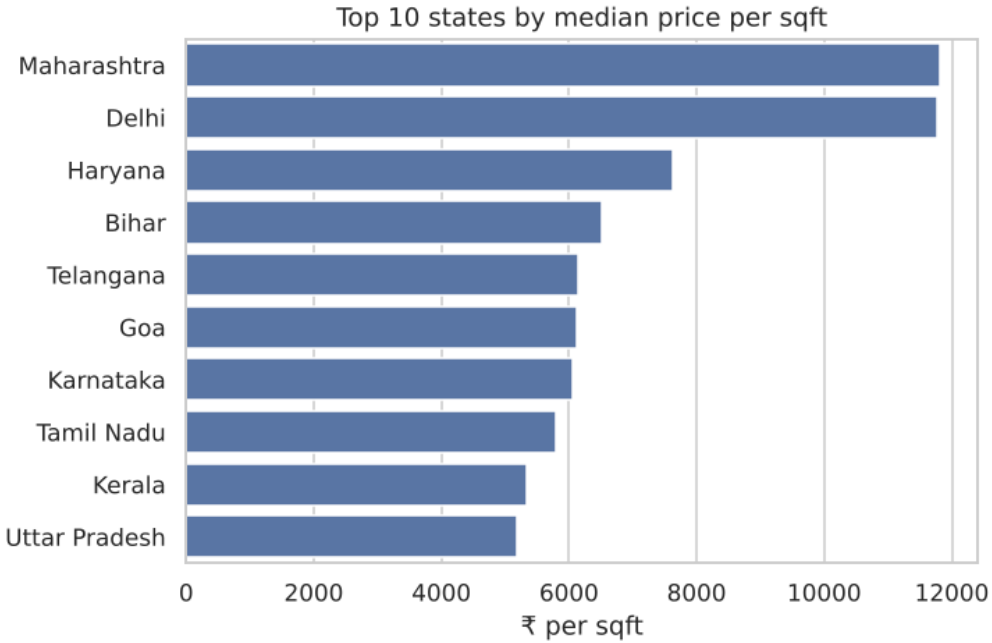

Median price per sqft by city (cities with 15+ listings)
Mumbai
Navi-Mumbai
Thane
New-Delhi
Bhiwandi
Gurgaon
Pune
Chennai
Noida
Hyderabad
Bangalore
Ahmedabad
Lucknow
Ghaziabad
Kochi
Coimbatore
Greater-Noida
Bhubaneswar
Visakhapatnam
Faridabad
Kolkata
Nagpur
Vadodara
Jaipur
Chandigarh
Gandhinagar
Indore
Bhopal
0
5000
10000
15000
20000
25000
Median price per sqft (₹)

City	listings	median_price_lakh	median_pps
Mumbai	195	185.0	26538.0
Navi-Mumbai	41	82.0	13004.0
Thane	101	72.0	11875.0
New-Delhi	20	164.0	11762.0
Bhiwandi	18	56.0	11136.0
Gurgaon	101	160.0	8972.0
Pune	184	66.0	8552.0
Chennai	80	74.0	6194.0
Noida	149	76.0	6153.0
Hyderabad	120	105.0	6138.0

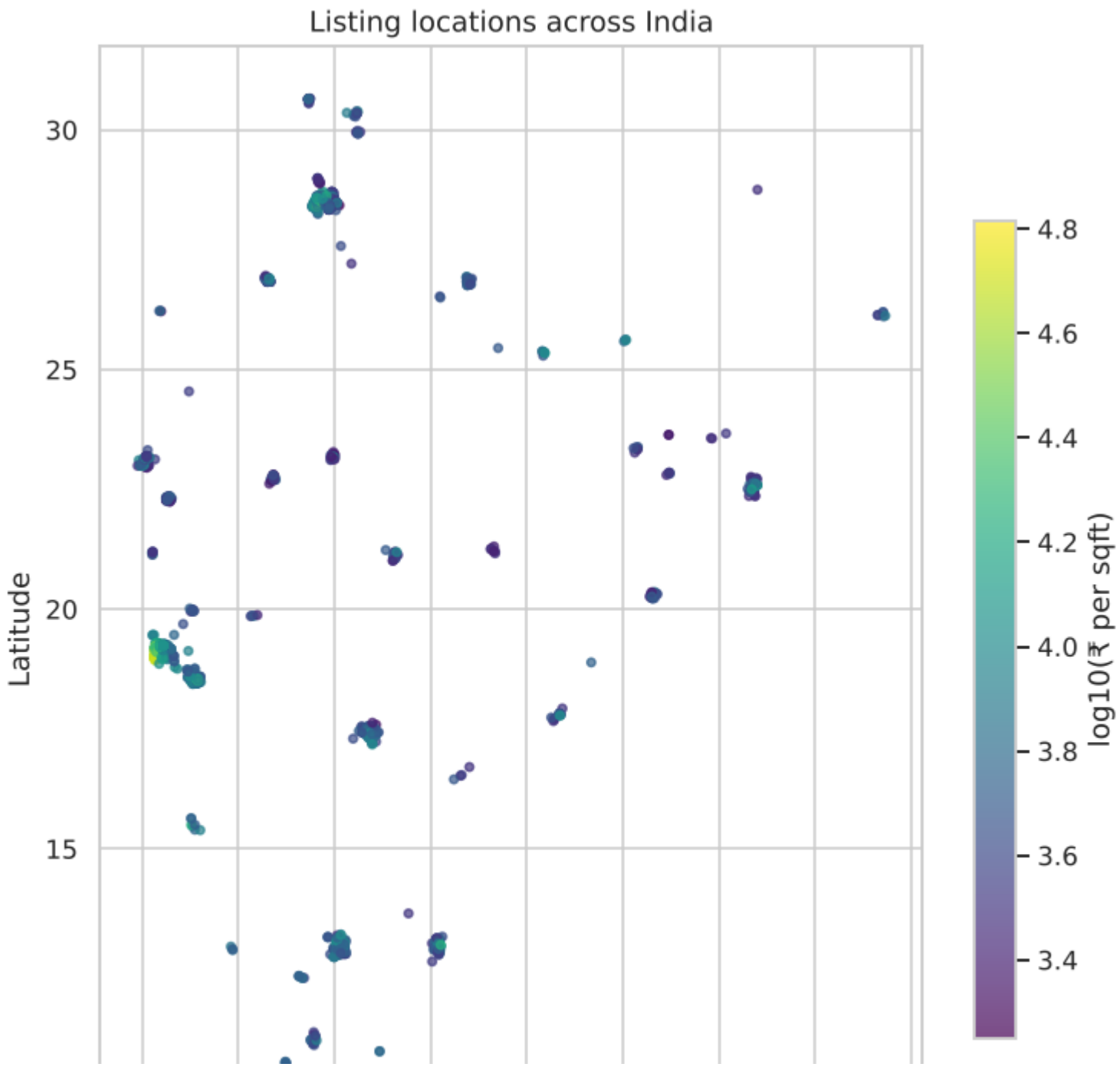
Observation: Mumbai is in a league of its own at about ₹26,500/sqft, roughly double anywhere else. Next come the wider Mumbai region (Navi Mumbai, Thane, Bhiwandi) and New Delhi at ₹11,000–13,000/sqft, then Gurgaon and Pune. Comparing price **per sqft** instead of total price removes the effect of property size, so cities can be compared fairly.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
order = sorted(df['Tier'].unique())
sns.boxplot(data=df, x='Tier', y='Price per sqft', order=order, ax=axes[0], showfliers=False, color='#64B5CD')
axes[0].set_title('Price per sqft by city tier')
state = df.groupby('State/Territory')['Price per sqft'].median().sort_values(ascending=False).head(10)
sns.barplot(x=state.values, y=state.index, ax=axes[1], color='#4C72B0')
axes[1].set_title('Top 10 states by median price per sqft'); axes[1].set_xlabel('₹ per sqft'); axes[1].set_ylabel('')
save(fig, 're_05_tier_state')
```





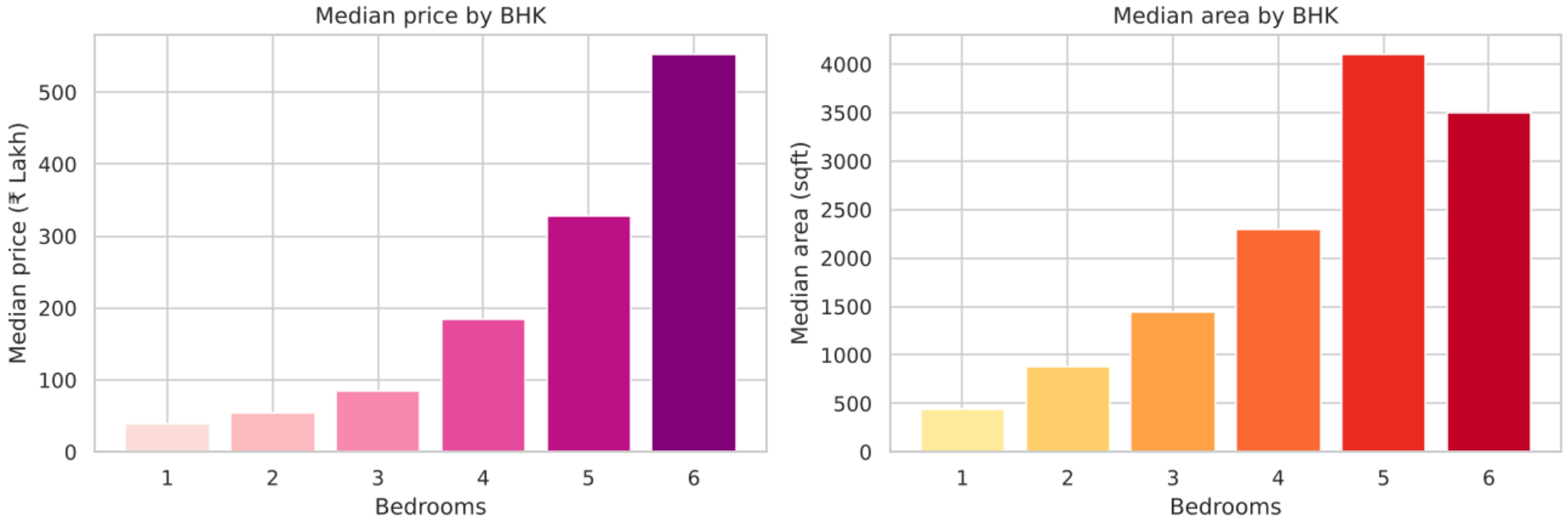

```
geo = df.dropna(subset=['Latitude', 'Longitude'])
geo = geo[geo['Latitude'].between(6, 37) & geo['Longitude'].between(68, 98)]
fig, ax = plt.subplots(figsize=(7, 8))
sc = ax.scatter(geo['Longitude'], geo['Latitude'], c=np.log10(geo['Price per sqft']), cmap='viridis', s=12, alpha=0.7)
cb = plt.colorbar(sc, ax=ax, shrink=0.7); cb.set_label('log10(₹ per sqft)')
ax.set_xlabel('Longitude'); ax.set_ylabel('Latitude'); ax.set_title('Listing locations across India')
save(fig, 're_06_geo_map')
```


Listing locations across India
Latitude
30
25
20
15
log10(₹ per sqft)
4.8
4.6
4.4
4.2
4.0
3.8
3.6
3.4

Observation: after fixing the swapped coordinates, the listings line up with the locations of the Indian cities they belong to, clustered around the major metro regions. The highest prices per sqft (the brightest points) are concentrated around Mumbai.

3. Property attributes

```
bed = df[df['Bedrooms'] <= 6].groupby('Bedrooms')[['Price (Lakh)', 'Area(sqft)']].median()
fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
axes[0].bar(bed.index, bed['Price (Lakh)'], color=sns.color_palette('RdPu', len(bed)))
axes[0].set_xlabel('Bedrooms'); axes[0].set_ylabel('Median price (₹ Lakh)'); axes[0].set_title('Median price by BHK')
axes[1].bar(bed.index, bed['Area(sqft)'], color=sns.color_palette('YlOrRd', len(bed)))
axes[1].set_xlabel('Bedrooms'); axes[1].set_ylabel('Median area (sqft)'); axes[1].set_title('Median area by BHK')
save(fig, 're_07_bhk_price_area')
bed.round(0)
```



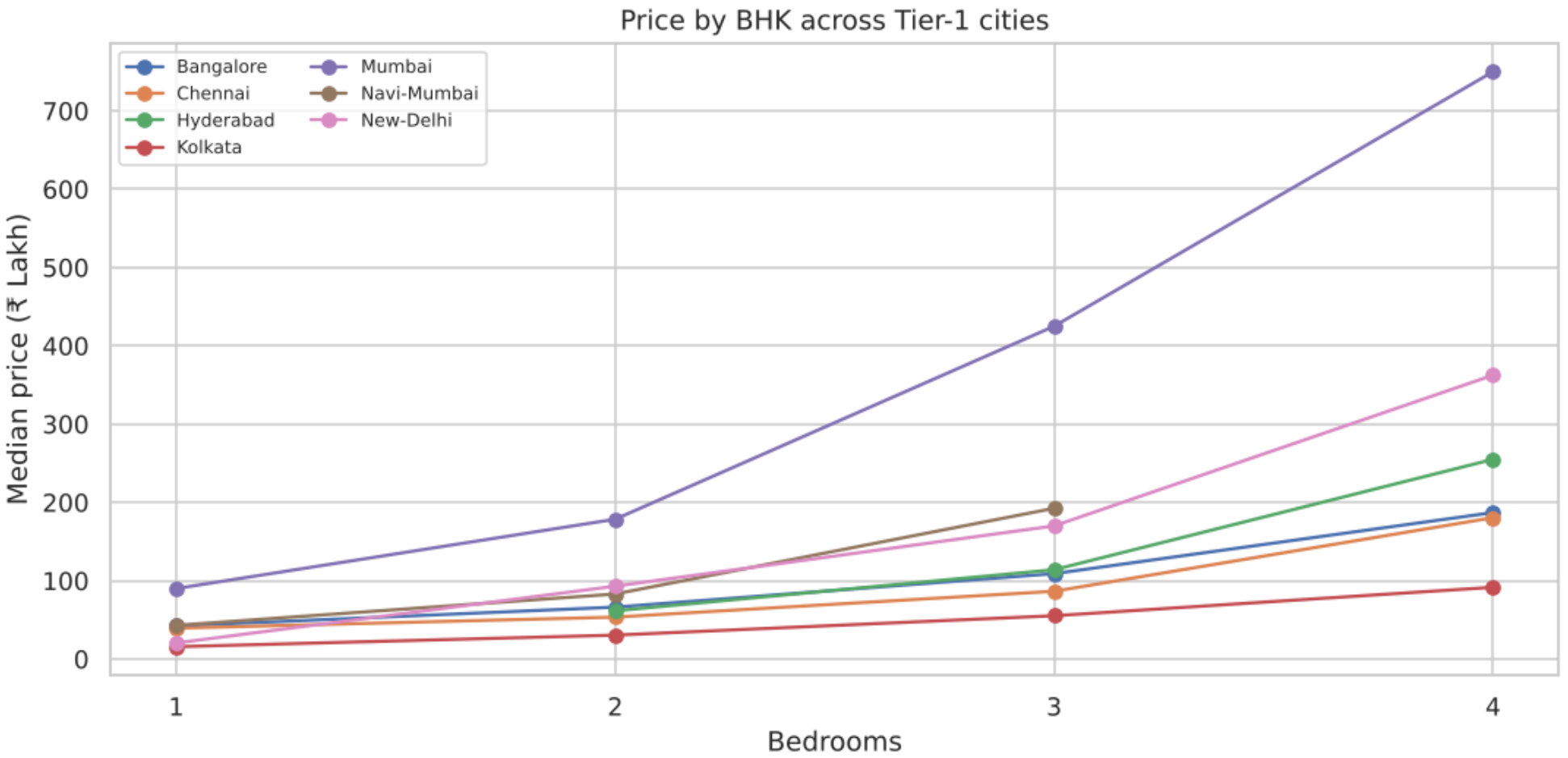
Bedrooms	Price (Lakh)	Area(sqft)
1	40.0	444.0
2	55.0	886.0
3	85.0	1440.0
4	184.0	2303.0
5	328.0	4102.0
6	552.0	3500.0

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
sns.barplot(data=df, x='Furnishing', y='Price (Lakh)', hue='Transaction', estimator='median', errorbar=None,
            order=['Unfurnished', 'Semi-Furnished', 'Furnished'], ax=axes[0])
axes[0].set_title('Median price by furnishing and transaction type')
sns.barplot(data=df, x='Property Type', y='Price per sqft', estimator='median', errorbar=None, ax=axes[1], color='#55A868')
axes[1].set_title('Median price per sqft by property type'); axes[1].tick_params(axis='x', rotation=15)
save(fig, 're_08_furnishing_type')
```




```
tier1 = df[(df['Tier'] == 'Tier1') & df['Bedrooms'].between(1, 4)]
pivot = tier1.pivot_table(index='Bedrooms', columns='City', values='Price (Lakh)', aggfunc='median')
fig, ax = plt.subplots(figsize=(10, 5))
pivot.plot(marker='o', ax=ax)
```

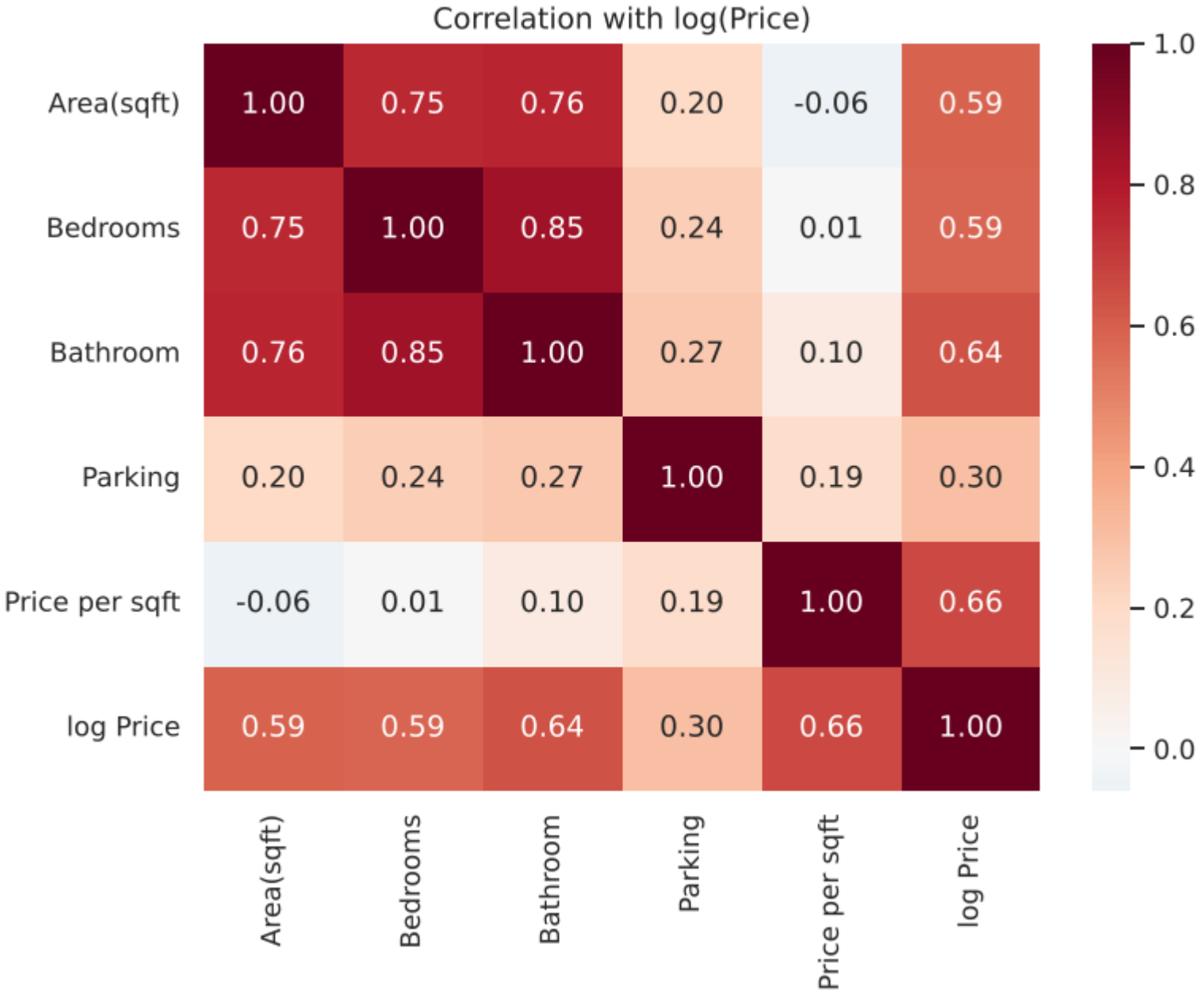
```
ax.set_ylabel('Median price (₹ Lakh)'); ax.set_xticks([1, 2, 3, 4])
ax.set_title('Price by BHK across Tier-1 cities'); ax.legend(ncol=2, fontsize=8)
save(fig, 're_09_tier1_bhk_lines')
pivot.round(0)
```



City Bedrooms	Bangalore	Chennai	Hyderabad	Kolkata	Mumbai	Navi-Mumbai	New-Delhi
1	42.0	39.0	NaN	16.0	90.0	43.0	20.0
2	66.0	54.0	62.0	30.0	178.0	83.0	93.0
3	109.0	86.0	114.0	55.0	425.0	192.0	170.0
4	187.0	180.0	255.0	92.0	750.0	NaN	362.0

4. Correlations

```
num = df[['Price', 'Area(sqft)', 'Bedrooms', 'Bathroom', 'Parking', 'Price per sqft']].copy()
num['log Price'] = np.log(num.pop('Price'))
fig, ax = plt.subplots(figsize=(7.5, 6))
sns.heatmap(num.corr(), annot=True, fmt='.2f', cmap='RdBu_r', center=0, ax=ax)
ax.set_title('Correlation with log(Price)')
save(fig, 're_10_correlation')
```


Correlation with log(Price)
Area(sqft)
Bedrooms
Bathroom
Parking
Price per sqft
log Price
1.00 0.75 0.76 0.20 -0.06 0.59
0.75 1.00 0.85 0.24 0.01 0.59
0.76 0.85 1.00 0.27 0.10 0.64
0.20 0.24 0.27 1.00 0.19 0.30
-0.06 0.01 0.10 0.19 1.00 0.66
0.59 0.59 0.64 0.30 0.66 1.00
Area(sqft)
Bedrooms
Bathroom
Parking
Price per sqft
log Price
1.0
0.8
0.6
0.4
0.2
0.0

Key EDA takeaways

1. **Location matters most.** Mumbai's median price per sqft (~₹26,500) is about double that of any other city. Tier-1 cities (median ~₹7,900/sqft) cost more than Tier-2 (~₹6,000) and smaller cities (~₹3,700–4,700).
2. **Size drives the total price.** Bathrooms (0.64), area (0.59) and bedrooms (0.59) correlate strongly with log(price). The median price jumps from ₹55 Lakh for a 2 BHK to ₹85 Lakh for a 3 BHK and ₹184 Lakh for a 4 BHK.
3. **New properties list higher than resale**, and **semi-furnished** homes carry the highest median price in both groups. Fully furnished homes show little premium over unfurnished.
4. **Prices are log-normal.** Modelling log(price) keeps luxury listings from dominating the error.

04 · Price Prediction Model

Predicts a property's listing price from its size, location and attributes.

Fixes compared with the original modelling code

Issue in original	Why it matters	Fix
`Price per sqft` used as an input feature	It is calculated **from** price, so the model was effectively given the answer (target leakage). This inflated test R^2 to 0.98.	Removed from the features
Raw price as target	Heavily skewed, so a few crore-rupee homes dominate the error	Model `log(price)` and convert predictions back to ₹
One random train/test split	A single split can be lucky or unlucky	5-fold cross-validation plus a held-out test set
Raw area with a log target	Linear models extrapolate exponentially for very large homes, so one fold scored $R^2 < 0$	Use `log(area)` (log-log model)
Swapped latitude/longitude, typo prices	Wrong geography and absurd prices	Fixed in notebook 02
`normalize=` and `criterion='mse'` arguments	Removed in recent scikit-learn, so the code crashes	Replaced with current equivalents

```
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', 30)
sns.set_theme(style='whitegrid')
plt.rcParams['figure.dpi'] = 110

ROOT = Path('..')
IMG_DIR = ROOT / 'images'
IMG_DIR.mkdir(exist_ok=True)

def save(fig, name):
    fig.tight_layout()
    fig.savefig(IMG_DIR / f'{name}.png', bbox_inches='tight')
    plt.show()
```

```
from sklearn.model_selection import train_test_split, KFold, cross_validate
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_absolute_percentage_error
import joblib

RANDOM_STATE = 42
```

1. Load data and choose features

```
df = pd.read_csv(ROOT / 'data/processed/magicbricks_cleaned.csv')
df['Log Area'] = np.log(df['Area(sqft)'])   # price grows multiplicatively with size

numeric = ['Log Area', 'Bedrooms', 'Bathroom', 'Parking', 'Latitude', 'Longitude']
categorical = ['City', 'Tier', 'State/Territory', 'Property Type', 'Transaction', 'Furnishing', 'Status']
features = numeric + categorical

X = df[features]
y = df['Price']
print('Rows:', len(df), '| Features:', len(features))
assert 'Price per sqft' not in features  # guard against target leakage
```

```
Rows: 2004 | Features: 13
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=RANDOM_STATE)
print('Train:', X_train.shape, '| Test:', X_test.shape)
```

```
Train: (1603, 13) | Test: (401, 13)
```

2. Preprocessing pipeline

- **Numeric columns:** fill missing coordinates with the median, then standardise.
- **Categorical columns:** one-hot encode, ignoring categories not seen during training.

Everything is wrapped in a scikit-learn `Pipeline`, so the exact same transformations are applied at prediction time.

```
preprocess = ColumnTransformer([
    ('num', Pipeline([('impute', SimpleImputer(strategy='median')), ('scale', StandardScaler())]), numeric),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical),
])

def make_model(regressor):
    # Fit on log(price); predictions are converted back to rupees automatically
    return TransformedTargetRegressor(
        regressor=Pipeline([('prep', preprocess), ('model', regressor)]),
        func=np.log, inverse_func=np.exp)

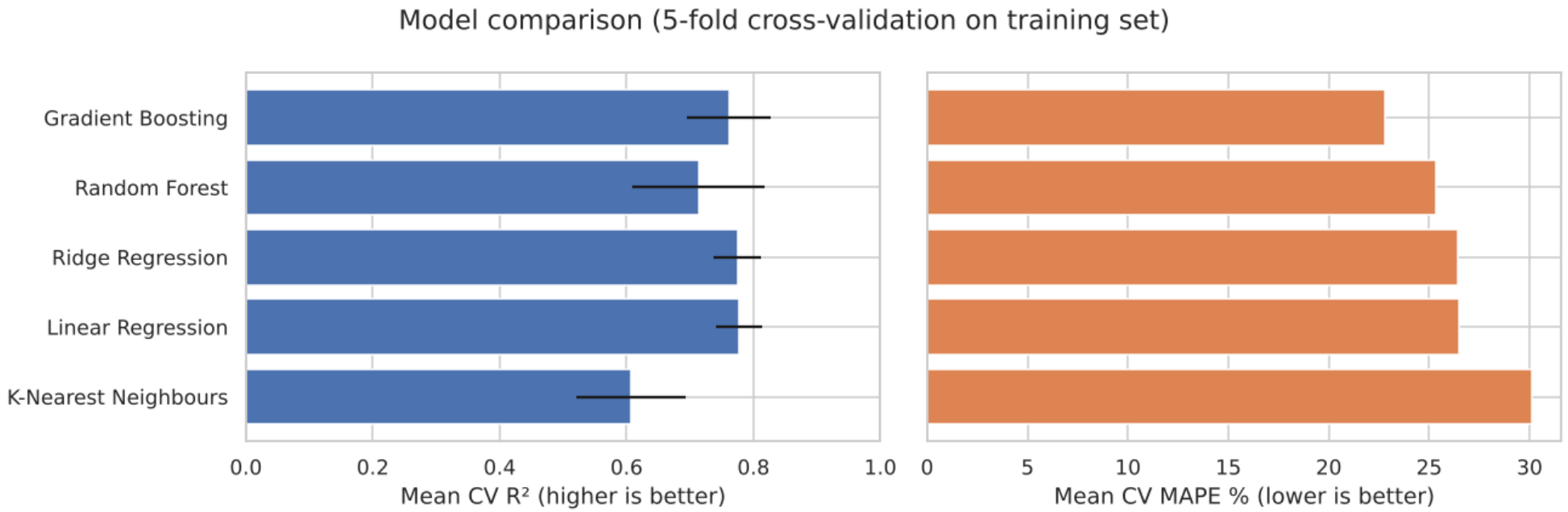
candidates = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'K-Nearest Neighbours': KNeighborsRegressor(n_neighbors=7, weights='distance'),
    'Random Forest': RandomForestRegressor(n_estimators=300, min_samples_leaf=2, random_state=RANDOM_STATE, n_jobs=-1),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=400, learning_rate=0.05, max_depth=4,
                                                   subsample=0.8, random_state=RANDOM_STATE),
}
```

3. Compare models with 5-fold cross-validation

```
cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
rows = []
for name, reg in candidates.items():
    scores = cross_validate(make_model(reg), X_train, y_train, cv=cv,
                            scoring={'r2': 'r2', 'mape': 'neg_mean_absolute_percentage_error'})
    rows.append({'Model': name,
                 'CV R²': scores['test_r2'].mean(),
                 'CV R² std': scores['test_r2'].std(),
                 'CV MAPE %': -scores['test_mape'].mean() * 100})
cv_results = pd.DataFrame(rows).set_index('Model').sort_values('CV MAPE %')
cv_results.round(3)
```

Model	CV R²	CV R² std	CV MAPE %
Gradient Boosting	0.761	0.066	22.770
Random Forest	0.713	0.105	25.360
Ridge Regression	0.775	0.036	26.456
Linear Regression	0.777	0.037	26.518
K-Nearest Neighbours	0.608	0.086	30.087

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.barh(cv_results.index, cv_results['CV R²'], xerr=cv_results['CV R² std'], color='#4C72B0')
ax1.invert_yaxis(); ax1.set_xlim(0, 1); ax1.set_xlabel('Mean CV R² (higher is better)')
ax2.barh(cv_results.index, cv_results['CV MAPE %'], color='#DD8452')
ax2.invert_yaxis(); ax2.set_xlabel('Mean CV MAPE % (lower is better)'); ax2.set_yticklabels([])
fig.suptitle('Model comparison (5-fold cross-validation on training set)')
save(fig, 're_11_model_comparison')
```



The models are ranked by **cross-validated MAPE** (mean absolute percentage error). Prices range from ₹9 Lakh to ₹24 Cr, and raw-scale R^2 is dominated by the few most expensive homes. MAPE weights a ₹30 Lakh flat and a ₹3 Cr villa equally, which is what a buyer cares about: *how far off is the estimate for my home?*

4. Evaluate the best model on the held-out test set

```
best_name = cv_results.index[0]
best_model = make_model(candidates[best_name]).fit(X_train, y_train)
pred = best_model.predict(X_test)

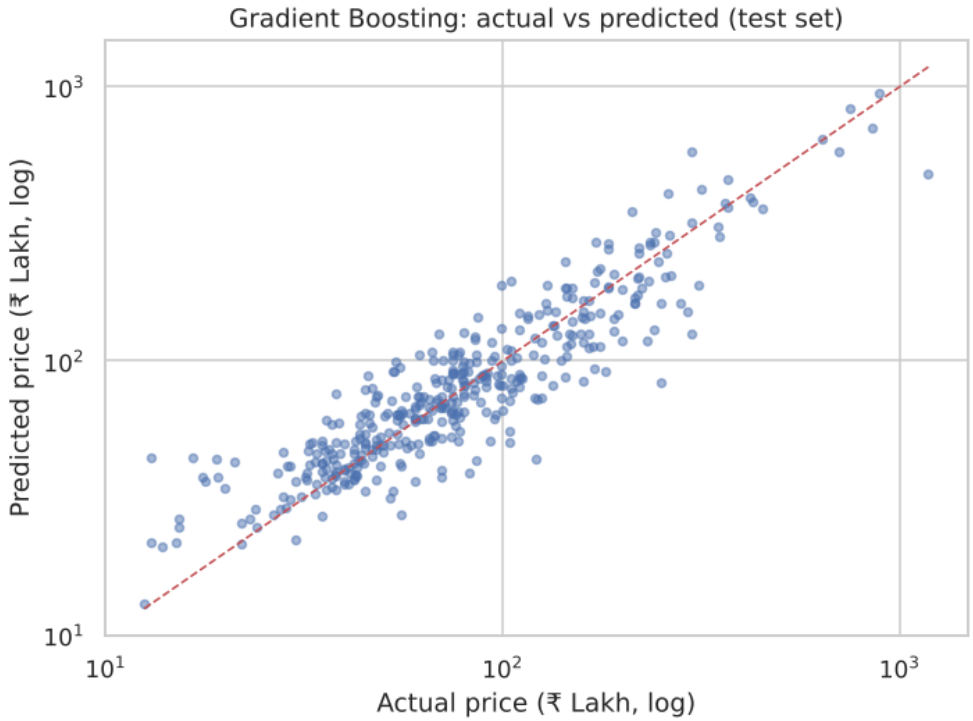
test_metrics = {
```

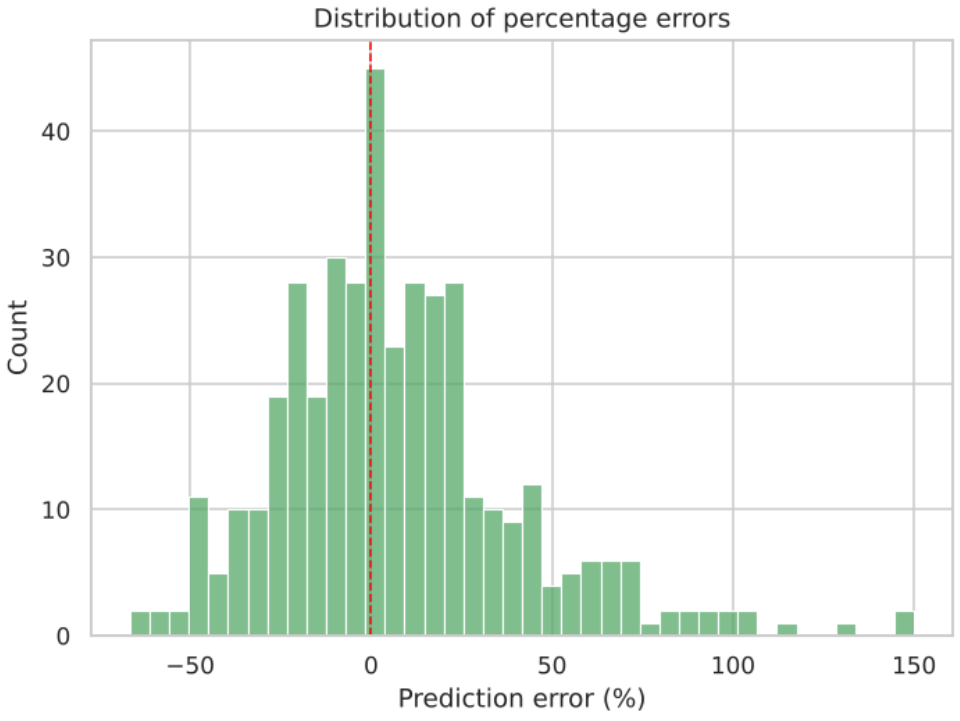
```
    'R²': r2_score(y_test, pred),
    'R² on log scale': r2_score(np.log(y_test), np.log(pred)),
    'MAE (₹ Lakh)': mean_absolute_error(y_test, pred) / 1e5,
    'Median abs. error (₹ Lakh)': np.median(np.abs(y_test - pred)) / 1e5,
    'MAPE %': mean_absolute_percentage_error(y_test, pred) * 100,
}
print('Best model:', best_name)
pd.Series(test_metrics).round(3)
```

```
Best model: Gradient Boosting

R²                            0.808
R² on log scale               0.826
MAE (₹ Lakh)                 25.492
Median abs. error (₹ Lakh)   13.759
MAPE %                       25.127
dtype: float64
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].scatter(y_test / 1e5, pred / 1e5, alpha=0.5, s=15)
lim = [y_test.min() / 1e5, y_test.max() / 1e5]
axes[0].plot(lim, lim, 'r--', lw=1); axes[0].set_xscale('log'); axes[0].set_yscale('log')
axes[0].set_xlabel('Actual price (₹ Lakh, log)'); axes[0].set_ylabel('Predicted price (₹ Lakh, log)')
axes[0].set_title(f'{best_name}: actual vs predicted (test set)')
pct_err = (pred - y_test) / y_test * 100
sns.histplot(pct_err.clip(-100, 150), bins=40, ax=axes[1], color='#55A868')
axes[1].axvline(0, color='red', ls='--', lw=1)
axes[1].set_xlabel('Prediction error (%)'); axes[1].set_title('Distribution of percentage errors')
save(fig, 're_12_actual_vs_predicted')
print(f'Share of test predictions within ±20% of actual price: {(pct_err.abs() <= 20).mean():.0%}')
```

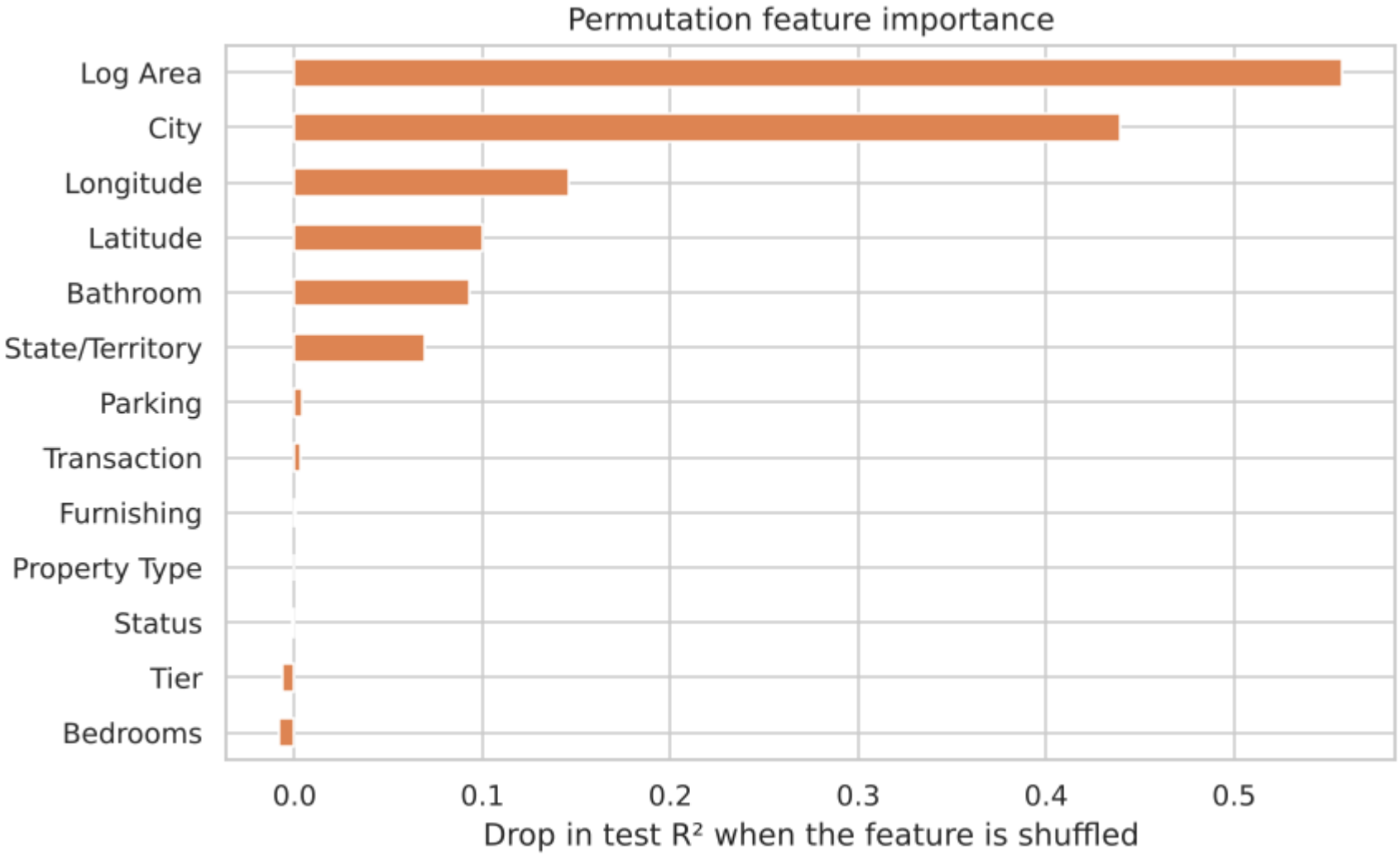





```
Share of test predictions within ±20% of actual price: 53%
```

5. What drives the prediction?

```
from sklearn.inspection import permutation_importance
perm = permutation_importance(best_model, X_test, y_test, n_repeats=10, random_state=RANDOM_STATE,
                              scoring='r2', n_jobs=-1)
importance = pd.Series(perm.importances_mean, index=features).sort_values()
fig, ax = plt.subplots(figsize=(8, 5))
importance.plot(kind='barh', ax=ax, color='#DD8452')
ax.set_xlabel('Drop in test R² when the feature is shuffled'); ax.set_title('Permutation feature importance')
save(fig, 're_13_feature_importance')
importance.sort_values(ascending=False).round(3)
```


Permutation feature importance
Log Area
City
Longitude
Latitude
Bathroom
State/Territory
Parking
Transaction
Furnishing
Property Type
Status
Tier
Bedrooms
0.0
0.1
0.2
0.3
0.4
0.5
Drop in test R² when the feature is shuffled

```
Log Area           0.558
City               0.439
Longitude          0.146
Latitude           0.100
Bathroom           0.093
State/Territory    0.069
Parking            0.004
Transaction        0.003
Furnishing         0.000
Property Type     -0.000
Status            -0.001
Tier              -0.006
Bedrooms          -0.008
dtype: float64
```

Permutation importance measures how much accuracy drops when one input column is randomly shuffled. Unlike tree-based impurity importance, it works on the original columns (before one-hot encoding), so the results are easy to read.

6. Leakage check — what the original approach would have reported

```python
leaky = make_model(candidates[best_name])
leaky_num = numeric + ['Price per sqft']
leaky.regressor.set_params(prep=ColumnTransformer([
    ('num', Pipeline([('impute', SimpleImputer(strategy='median')), ('scale', StandardScaler())]), leaky_num),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical)]))
Xl_train, Xl_test = train_test_split(df[leaky_num + categorical], test_size=0.2, random_state=RANDOM_STATE)
leaky.fit(Xl_train, y_train)
leaky_pred = leaky.predict(Xl_test)
print(pd.DataFrame({
    'R²': [r2_score(y_test, leaky_pred), test_metrics['R²']],
    'R² (log scale)': [r2_score(np.log(y_test), np.log(leaky_pred)), test_metrics['R² on log scale']],
    'MAPE %': [mean_absolute_percentage_error(y_test, leaky_pred) * 100, test_metrics['MAPE %']],
}, index=["WITH leaked 'Price per sqft' (not usable)", 'Without leakage (honest)']).round(3))
```

```
                                        R²  R² (log scale)  MAPE %
WITH leaked 'Price per sqft' (not usable)  0.994           0.998   2.556
Without leakage (honest)                   0.808           0.826  25.127
```

`Price per sqft` is unknown for a new property; it's exactly what we're trying to estimate. A model that depends on it can't be used in practice, so the honest score is the one reported above.

7. Train on all data, save and use the model

```
final_model = make_model(candidates[best_name]).fit(X, y)
(ROOT / 'models').mkdir(exist_ok=True)
joblib.dump(final_model, ROOT / 'models/price_model.joblib')
print('Saved model to models/price_model.joblib')
```

```
Saved model to models/price_model.joblib
```

```
def predict_price(area_sqft, bedrooms, bathrooms, city, parking=1, property_type='Flat',
                  transaction='New Property', furnishing='Unfurnished', status='Ready to Move'):
    """Predict the listing price (₹) for one property. Coordinates default to the city's median."""
    city_rows = df[df['City'] == city]
    if city_rows.empty:
        raise ValueError(f'Unknown city: {city}')
    row = pd.DataFrame([{
        'Log Area': np.log(area_sqft), 'Bedrooms': bedrooms, 'Bathroom': bathrooms, 'Parking': parking,
        'Latitude': city_rows['Latitude'].median(), 'Longitude': city_rows['Longitude'].median(),
        'City': city, 'Tier': city_rows['Tier'].iloc[0], 'State/Territory': city_rows['State/Territory'].iloc[0],
        'Property Type': property_type, 'Transaction': transaction, 'Furnishing': furnishing, 'Status': status,
    }])
    return float(final_model.predict(row[features])[0])

examples = [
    dict(area_sqft=1000, bedrooms=2, bathrooms=2, city='Pune'),
    dict(area_sqft=1000, bedrooms=2, bathrooms=2, city='Mumbai'),
    dict(area_sqft=1500, bedrooms=3, bathrooms=3, city='Bangalore', furnishing='Semi-Furnished'),
    dict(area_sqft=1200, bedrooms=2, bathrooms=2, city='Ahmedabad', transaction='Resale'),
]
pd.DataFrame([{**e, 'Predicted price (₹ Lakh)': round(predict_price(**e) / 1e5, 1)} for e in examples])
```

	area_sqft	bedrooms	bathrooms	city	Predicted price (₹ Lakh)	furnishing	transaction
0	1000	2	2	Pune	75.1	NaN	NaN
1	1000	2	2	Mumbai	215.2	NaN	NaN
2	1500	3	3	Bangalore	110.3	Semi-Furnished	NaN
3	1200	2	2	Ahmedabad	59.3	NaN	Resale

Sanity check: the same 1,000 sqft 2 BHK is predicted at about ₹75 Lakh in Pune and ₹215 Lakh in Mumbai. That's roughly ₹7,500 vs ₹21,500 per sqft, in line with each city's median price per sqft from the EDA.

Summary

	Value
Final model	Gradient Boosting on log(price)
Cross-validated MAPE	~23%
Test R^2 (₹ scale / log scale)	0.81 / 0.83
Test MAPE	~25%; about half of predictions within ±20%

- **Size and location dominate.** `log(area)` and `City`, followed by the exact coordinates and the number of bathrooms, carry nearly all the predictive power. Bedrooms add almost nothing once area and bathrooms are known, because the three are highly correlated.
- **Linear/Ridge regression score a similar R^2** but a higher percentage error. Gradient Boosting captures the non-linear city-by-size effects better.
- **Leakage matters.** Including `Price per sqft` would have reported a fake R^2 of 0.99. The honest model is less impressive on paper, but it can price an unseen property.
- **Limitations:** about 2,000 listings across 59 cities, so smaller cities have few examples. These are asking prices, not sale prices, and amenities and floor level aren't modelled yet.